

Experiment 1

AIM

To implement a Gym environment for solving a Grid World navigation problem where the agent reaches the goal while avoiding obstacles by defining the state space, action space, reward function, and constraints

Algorithm

1. Import the Gymnasium library.
2. Create the FrozenLake environment.
3. Reset the environment to the initial state.
4. Select random actions for the agent.
5. Observe the next state, reward, and episode status.
6. Repeat until the goal is reached or the episode ends.
7. Display the state transitions and rewards.

Program

```
import gymnasium as gym

env = gym.make("FrozenLake-v1", is_slippery=False)

state, info = env.reset()

print("\n==== Grid World Navigation =====\n")

print(f"{'Step':<6}{'State':<8}{'Action':<8}{'Reward':<8}")

print("-"*35)

for step in range(15):

    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    print(f"{'step+1':<6}{'state':<8}{'action':<8}{'reward':<8}")

    state = next_state

    if terminated or truncated:

        break

env.close()
```

Output

```
===== Grid World Navigation =====
```

Step	State	Action	Reward
1	0	0	0
2	0	2	0
3	1	3	0
4	1	2	0
5	2	0	0
6	1	2	0
7	2	3	0
8	2	1	0
9	6	0	0

Result

Thus, the Grid World navigation problem was successfully implemented using the Gymnasium FrozenLake environment. The agent interacted with the environment by selecting actions, observing rewards, and navigating toward the goal while avoiding obstacles.

Experiment 2**AIM**

To implement an ϵ -greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials and compare the effect of ϵ values (0.1, 0.3, and 0.5) on exploration and exploitation

Algorithm

1. Import the NumPy library.
2. Define the reward probabilities for each advertisement.
3. Set ϵ values as 0.1, 0.3, and 0.5.
4. Run the ϵ -greedy algorithm for 500 trials.
5. Calculate the cumulative reward for each ϵ value.
6. Display the cumulative rewards.
7. Compare the performance of different ϵ values.

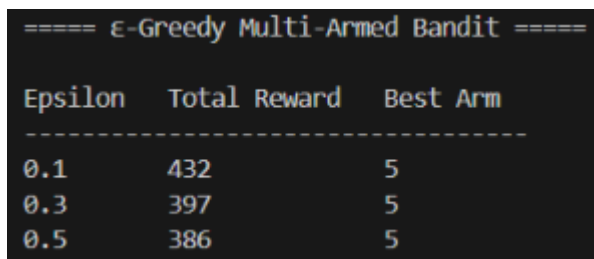
Program

```
import numpy as np
```

```
true_rewards = [0.2, 0.5, 0.7, 0.4, 0.9]
```

```
n_arms = len(true_rewards)
episodes = 500
epsilons = [0.1, 0.3, 0.5]
print("\n=====  $\epsilon$ -Greedy Multi-Armed Bandit =====\n")
print(f"{'Epsilon':<10}{'Total Reward':<15}{'Best Arm':<10}")
print("-" * 35)
for epsilon in epsilons:
    Q = np.zeros(n_arms)
    N = np.zeros(n_arms)
    total_reward = 0
    for _ in range(episodes):
        if np.random.rand() < epsilon:
            action = np.random.randint(n_arms)
        else:
            action = np.argmax(Q)
        reward = 1 if np.random.rand() < true_rewards[action] else 0
        total_reward += reward
        N[action] += 1
        Q[action] += (reward - Q[action]) / N[action]
    best_arm = np.argmax(Q) + 1
    print(f"{'epsilon':<10}{'total_reward':<15}{'best_arm':<10}")
```

Output



Epsilon	Total Reward	Best Arm
0.1	432	5
0.3	397	5
0.5	386	5

Result

Thus, the ϵ -greedy Multi-Armed Bandit algorithm was successfully implemented. Different ϵ values affected the balance between exploration and exploitation, and the algorithm identified the best-performing advertisement while maximizing cumulative rewards under 500 trials.

Experiment 3

AIM

To design a Markov Decision Process (MDP) for an autonomous robot that reaches a destination while minimizing energy consumption by defining states, actions, transition probabilities, rewards, and energy constraints.

Algorithm

1. Define the states and actions of the robot.
2. Initialize the robot with a fixed energy level.
3. Select the best action based on the reward.
4. Move to the next state and reduce the energy.
5. Repeat until the goal is reached.
6. Display the robot's path and remaining energy.

Program

```
states = {  
    "Start": {"Forward": ("Room1", 5, 5)},  
    "Room1": {"Forward": ("Room2", 8, 5)},  
    "Room2": {"Forward": ("Goal", 15, 5)},  
    "Goal": {}  
}  
  
energy = 30  
  
current_state = "Start"  
  
print("\n===== Autonomous Robot Navigation =====\n")  
print(f"{'Step':<6}{ 'State':<10}{ 'Action':<12}{ 'Energy':<10}")  
  
print("-" * 42)  
  
step = 1
```

```
while current_state != "Goal" and energy > 0:
    action = max(states[current_state], key=lambda x: states[current_state][x][1])
    next_state, reward, cost = states[current_state][action]
    energy -= cost
    print(f"step:<6}{current_state:<10}{action:<12}{energy:<10}")
    current_state = next_state
    step += 1
print("\nDestination Reached :", current_state == "Goal")
```

Output

```
===== Autonomous Robot Navigation =====
Step  State      Action      Energy
-----
1     Start      Forward     25
2     Room1       Forward     20
3     Room2       Forward     15
Destination Reached : True
```

Result

Thus, the Markov Decision Process for an autonomous robot was successfully implemented, and the robot reached the destination while minimizing energy consumption.

Experiment 4

AIM

To implement a Value Iteration algorithm to determine the optimal policy for a constrained Grid World using Bellman equations.

Algorithm

1. Define the states and rewards.
2. Initialize the state values.
3. Apply the Bellman equation iteratively.
4. Update the values of all states.
5. Repeat for a fixed number of iterations.

6. Display the optimal state values.

Program

```
gamma = 0.9
states = ["A", "B", "C", "Goal"]
values = {
    "A": 0,
    "B": 0,
    "C": 0,
    "Goal": 20
}
rewards = {
    "A": -2,
    "B": -1,
    "C": -2,
    "Goal": 20
}
transitions = {
    "A": "B",
    "B": "C",
    "C": "Goal"
}
print("\n===== Value Iteration =====\n")
print(f"{'Iteration':<10}{'State':<10}{'Value':<10}")
print("-" * 32)
for iteration in range(5):
    for state in ["C", "B", "A"]:
        next_state = transitions[state]
        values[state] = rewards[state] + gamma * values[next_state]
```

for state in states:

```
print(f"{iteration+1:<10}{state:<10}{values[state]:<10.2f}")
```

```
print("\nOptimal Policy : A → B → C → Goal")
```

Output

```
===== Value Iteration =====
```

Iteration	State	Value
1	A	10.06
1	B	13.40
1	C	16.00
1	Goal	20.00
2	A	10.06
2	B	13.40
2	C	16.00
2	Goal	20.00
3	A	10.06
3	B	13.40
3	C	16.00
3	Goal	20.00
4	A	10.06
4	B	13.40
4	C	16.00
4	Goal	20.00
5	A	10.06
5	B	13.40
5	C	16.00
5	Goal	20.00

Optimal Policy : A → B → C → Goal

Result

Thus, the Value Iteration algorithm was successfully implemented, and the optimal policy was obtained using the Bellman equation

Experiment 5

AIM

To design a Reinforcement Learning framework for a warehouse robot that completes the maximum number of deliveries without exceeding a battery capacity of 30 minutes.

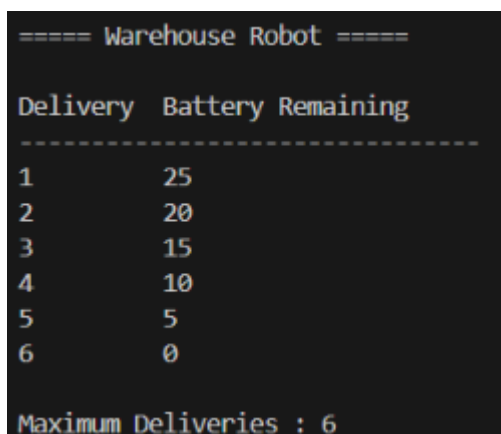
Algorithm

1. Initialize the battery capacity.
2. Set the delivery time for each task.

3. Perform deliveries while battery is available.
4. Reduce the battery after each delivery.
5. Display the completed deliveries and remaining battery.
6. Stop when the battery is exhausted.

Program

```
battery = 30
delivery_time = 5
deliveries = 0
print("\n===== Warehouse Robot =====\n")
print(f"{'Delivery':<10}{'Battery Remaining':<20}")
print("-" * 32)
while battery >= delivery_time:
    deliveries += 1
    battery -= delivery_time
    print(f"{'deliveries':<10}{'battery':<20}")
print("\nMaximum Deliveries :", deliveries)
```

Output

```
===== Warehouse Robot =====
Delivery  Battery Remaining
-----
1         25
2         20
3         15
4         10
5          5
6          0
Maximum Deliveries : 6
```

Result

Thus, the Reinforcement Learning framework for the warehouse robot was successfully implemented, enabling the robot to complete the maximum number of deliveries within the available battery capacity.